The PGE Guide

PGE (Pinto's Game Engine) is a small engine for making 2D games in Java. This guide starts with your first game and ends with you adding new features to the engine itself. Read it in order the first time. After that, use it to look things up.

1. Getting started
2. Your first game
3. How a game runs
4. Pictures, animation and text
5. Sound and music
6. Levels, walls and the camera
7. Bigger games: scenes and game objects
8. Handing in your game
9. When something goes wrong
10. Inside the engine
11. Extending the engine

1. Getting started

You need Java 25 or newer, and it has to be a JDK (the version of Java that includes the compiler). To check which version you have, type this in a terminal:

```
java -version
```

Then go to the PGE folder and type:

```
java Build.java demo
```

This compiles the engine and all the games, and then starts the game in `games/demo`. The other games are called `platformer`, `platformer-scenes` and `starter`. Try them too.

`Build.java` does all the building. It is an ordinary Java program that calls the Java compiler, so you can open it and read how it works.

Using an editor

You can build and run every game from the terminal, so any text editor will do. But a Java editor helps a lot: it suggests method names as you type, underlines mistakes before you compile, jumps to the code of any method, and lets you stop a running game and look inside it (see "Using the debugger" below). The next sections explain how to set up the two most popular ones, Visual Studio Code and IntelliJ IDEA.

PGE has no build tool (such as Maven or Gradle) that an editor could read its setup from, so you tell the editor three things yourself:

1. **Which Java to use:** a JDK of version 25 or newer.
2. **Where the code is:** the folder `engine/src`, and the folder of the game you are working on, such as `games/mygame`. Only add one game folder at a time. Every game has its own class called `Main` (and some share other names, such as `Coin`), and two classes with the same name confuse the editor.
3. **Which folder the game runs in:** the game's own folder, such as `games/mygame`. Games load their files with paths like `"assets/hero.png"`, which only work when the game is started in its own folder. If you forget this, the game stops with `Could not find the image 'assets/...'`.

The settings files that both editors create (`.vscode/` and `.idea/`) are listed in `.gitignore`, so each person keeps their own and they are never committed.

Visual Studio Code

Visual Studio Code ("VS Code") is a free editor from Microsoft. It needs extensions to understand Java.

Install the Java extensions. Open the Extensions view (the icon with four squares on the left, or Cmd+Shift+X on a Mac, Ctrl+Shift+X on Windows), search for **Extension Pack for Java** (by Microsoft) and install it. It includes the Java language support, the debugger and a test runner.

Open the PGE folder with File > Open Folder. The first time, VS Code may ask whether you trust the folder's authors; answer yes.

Tell VS Code about the code and the JDK. Create a folder called `.vscode` in the PGE folder, and in it a file called `settings.json`:

```
{
    "java.project.sourcePaths": ["engine/src", "games/mygame"],
    "java.project.outputPath": "bin",
    "java.configuration.runtimes": [
        {
            "name": "JavaSE-25",
            "path": "/path/to/your/jdk-25",
            "default": true
        }
    ]
}
```

- `java.project.sourcePaths` lists the code folders. Change `games/mygame` to the game you are working on.
- `java.project.outputPath` is where VS Code puts the compiled classes. `bin` keeps them apart from the `out` folder that `Build.java` uses.
- `java.configuration.runtimes` says where the JDK is. To find the path, run `java -XshowSettings:properties -version` in a terminal and look for `java.home`. (With SDKMAN it is something like `/Users/you/.sdkman/candidates/java/25.0.4-tem`.)

After saving, the Java extension takes a moment to read the project; you can watch it in the status bar at the bottom. When it is done, open `games/mygame/Main.java` and type `Input.` inside `update()`: a list of its methods should appear.

Run the game. In the same `.vscode` folder, create `launch.json`:

```
{
    "version": "0.2.0",
    "configurations": [
        {
            "type": "java",
            "name": "Run mygame",
            "request": "launch",
            "mainClass": "Main",
            "cwd": "${workspaceFolder}/games/mygame"
        }
    ]
}
```

`cwd` ("current working directory") is the folder the game runs in, the third of the three things above. Now press F5 (or use Run > Start Debugging) to start the game.

Or run it with Build.java. You can also use the built-in terminal (View > Terminal) and type `java Build.java mygame`, exactly as in section 1. To make that a single key press, create `.vscode/tasks.json`:

```
{
    "version": "2.0.0",
    "tasks": [
        {
            "label": "Build and run mygame",
            "type": "shell",
            "command": "java Build.java mygame",
            "group": { "kind": "build", "isDefault": true }
        }
    ]
}
```

Then Cmd+Shift+B (Ctrl+Shift+B on Windows) builds and runs the game. This way uses exactly the same build as handing in the game, but it does not stop at breakpoints.

Switching to another game: change `games/mygame` in `settings.json` and `launch.json`. If the editor still shows errors from the old game, open the command palette (Cmd+Shift+P or Ctrl+Shift+P) and run **Java: Clean Java Language Server Workspace**.

IntelliJ IDEA

IntelliJ IDEA is a Java editor from JetBrains. Students can use all of its features for free through JetBrains' education program (search for "JetBrains student licence"), and PGE only needs the basic Java features.

Open the PGE folder with File > Open. IntelliJ sees that there is no build tool and opens it as a plain project.

Choose the JDK. Open File > Project Structure (Cmd+; on a Mac, Ctrl+Alt+Shift+S on Windows). Under **Project**, set **SDK** to a JDK of version 25 or newer. If none is listed, choose **Add SDK > JDK** and pick the JDK's folder, or **Download JDK** to let IntelliJ fetch one. Set **Language level** to "SDK default".

Mark the code folders. In the Project panel on the left, right-click `engine/src` and choose **Mark Directory as > Sources Root**. Do the same for your game's folder, such as `games/mygame`. The folders turn blue. While you are there, right-click `out` and `dist` (if they exist) and choose **Mark Directory as > Excluded**, so IntelliJ does not search through compiled files.

Create a run configuration. Open Run > Edit Configurations, click **+** and choose **Application**. Fill in:

Field	Value
Name	`mygame`
Main class	`Main`
Working directory	`$PROJECT_DIR$/games/mygame`
Module (or "Use classpath of module")	`PGE`

`$PROJECT_DIR$` is IntelliJ's name for the PGE folder. Click OK. Now the green triangle at the top of the window runs the game, and the bug icon next to it starts it in the debugger.

Or run it with Build.java. Open the built-in terminal (View > Tool Windows > Terminal) and type `java Build.java mygame`.

Switching to another game: right-click the old game folder and choose **Mark Directory as > Unmark as Sources Root**, mark the new one, and change the working directory in the run configuration.

Optional: every game at once. IntelliJ can hold several *modules* in one project, and each module has its own source folders. Make one module for the engine and one per game, and give every game module a dependency on the engine module (Project Structure > Modules > your game > Dependencies > **+** > Module Dependency). Then all games are open together without their `Main` classes getting in each other's way, and you make one run configuration per game. It takes ten minutes to set up, and is worth it once you work on more than one game.

Using the debugger

The debugger lets you pause a running game at any line and look at all its variables. It is the fastest way to find out why something does not behave as you expect.

1. Click in the margin to the left of a line number, for example on the first line of `update()`. A red dot appears: that is a *breakpoint*.
2. Start the game with the debugger: F5 in VS Code, or the bug icon in IntelliJ.
3. When the game reaches the breakpoint, it freezes, and the editor shows the line. The Variables panel shows the value of every field, such as `playerX` and `score`.
4. **Step Over** (F10 in VS Code, F8 in IntelliJ) runs one line and stops again. **Step Into** (F11 in VS Code, F7 in IntelliJ) goes inside the method called on that line, even into the engine's own code. **Continue** (F5 in VS Code; in IntelliJ, F9 on Windows or Cmd+Option+R on a Mac) runs until the next breakpoint. On a Mac laptop, the F keys may need the fn key.

A breakpoint in `update()` stops the game 60 times per second, so for anything that happens rarely, put it inside the `if` that you are curious about. You can also right-click a breakpoint and give it a condition, such as `score == 10`, so it only stops when that is true.

When the game continues after a long pause, the engine runs up to five updates in a row to catch up, and then carries on as normal (see "The game loop" in `Engine.java`), so it is safe to pause for as long as you like.

2. Your first game

Copy the folder `games/starter` and name the copy `games/mygame`. Then run it:

```
java Build.java mygame
```

A yellow square appears, and you can move it with the arrow keys. Now open `games/mygame/Main.java`. It has three methods, and the engine calls them for you:

Method	When the engine calls it	What you do in it
`init()`	once, when the game starts	load pictures and sounds, set things up
`update()`	60 times every second	move things, check the rules, read the keyboard
`draw(canvas)`	after every update	draw everything on the screen

Now try these changes. Make one change at a time and run the game after each one.

1. Make the square faster by changing `playerSpeed`.
2. Change the colours. Try `Color.PINK`, or write a colour as a number: `0xFF8800` is orange. The six digits are three pairs, for red (FF), green (88) and blue (00).
3. Stop the square from leaving the screen. Put this in `update()`, after the code that moves it:

   ```
   if (playerX < 0) {
       playerX = 0;
   }
   if (playerX > Engine.getWidth() - 16) {
       playerX = Engine.getWidth() - 16;
   }
   ```

   16 is the width of the square. Do the same for `playerY`, using `Engine.getHeight()`.
4. Add a coin to collect. Add two fields, `coinX` and `coinY`. Draw a circle there with `canvas.fillCircle(...)`. In `update()`, check whether the square is close to the coin. If it is, add a point and move the coin to a random place, for example with `coinX = Math.random() * 600;`.
5. Add a time limit. Count updates in a field: 60 updates make one second. Show the time left with `canvas.drawText(...)`, and end the game when it reaches zero.

You now have a small but complete game. The rest of this guide shows you how to make it look better, and how to keep the code organised as the game grows.

3. How a game runs

Update, then draw

The engine calls `update()` exactly 60 times per second, on every computer. After each update it calls `draw()`. Each method has its own job:

- `update()` changes the game: positions, scores, timers. It never draws.
- `draw()` shows the game. It never changes anything.

Because updates always come 60 times per second, `x = x + 2` always means "move 2 pixels per update", which is 120 pixels per second. You never have to measure time yourself.

Coordinates

The game picture is 640 pixels wide and 360 pixels tall. You choose this size in `Engine.run`. The point `(0, 0)` is the top-left corner. `x` gets bigger as you go right, and `y` gets bigger as you go down. So to move something up, you make its `y` smaller.

The window is bigger than the game picture. The engine blows the picture up 2, 3 or 4 times so that it fills the window, and each game pixel becomes a small square on the screen. Your code never needs to know how big the window is.

World and screen

Often the game world is bigger than the screen. The camera decides which part of the world you see. You draw the world (the player, the enemies, the level) with the camera on. You draw the score and the menus with the camera off, so they stay in the same place on the screen:

```
canvas.setCamera(cameraX, cameraY);   // the world
canvas.drawBitmap(hero, heroX, heroY);
canvas.resetCamera();                 // the screen
canvas.drawText("Score: " + score, 4, 4, Color.WHITE);
```

The `Camera` class (section 6) works out `cameraX` and `cameraY` for you.

Keyboard and mouse

You want to know	Write
Is the key being held down?	`Input.isDown(Key.LEFT)`
Was the key pressed just now?	`Input.wasPressed(Key.SPACE)`
Was the key let go just now?	`Input.wasReleased(Key.SPACE)`
Where is the mouse?	`Input.getPointerX()`, `Input.getPointerY()`
Was the mouse clicked just now?	`Input.wasPointerPressed()`

Use `isDown` for things that go on while the key is held, like walking. Use `wasPressed` for things that should happen once per key press, like jumping, shooting or choosing a menu item.

Some keys work in every game. F11 (or Alt+Enter) switches fullscreen on and off. F3 shows how many milliseconds each frame takes. Each frame can take up to 16.7 milliseconds, and the example games use about 2.

4. Pictures, animation and text

Loading pictures

Put your PNG files in the `assets` folder of your game, and load them like this:

```
Bitmap hero = Bitmap.load("assets/hero.png");
```

The see-through parts of the PNG are not drawn. Some older pixel art uses one background colour (often bright pink) instead of see-through pixels. You can load those too, by saying which colour to leave out: `Bitmap.load("assets/hero.png", 0xFF00FF)`.

Games usually keep lots of small pictures together in one image, called a *sprite sheet*. The pictures are all the same size and sit in a grid:

```
// each picture in the sheet is 16 by 16 pixels
SpriteSheet sheet = SpriteSheet.load("assets/sprites.png", 16, 16);

Bitmap hero = sheet.get(0);    // the first picture
Bitmap coin = sheet.get(8);    // count left to right, then go to the next row
```

Open `games/demo/assets/sprites.png` to see what a sprite sheet looks like. You can draw your own pictures with a pixel art editor. Piskel is free and runs in the browser. Aseprite costs a little money and has more features.

Drawing pictures

```
// normal
canvas.drawBitmap(hero, x, y);

// flipped left to right, for walking left
canvas.drawBitmap(hero, x, y, true, false);

// all one colour, for a "got hit" flash
canvas.drawSilhouette(hero, x, y, false, Color.WHITE);
```

The positions must be whole numbers (`int`). If you store positions as `double`, which gives smoother movement, turn them into `int` when you draw: `(int) heroX`.

Animation

An `Animation` is a list of pictures that are shown one after another, like a flip book:

```
// pictures 0 to 3 of the sheet, each one shown for 8 updates
Animation walk = new Animation(sheet.getFrames(0, 4), 8);

// in update():
walk.update();

// in draw():
canvas.drawBitmap(walk.getFrame(), x, y);
```

Text

```
canvas.drawText("Score: " + score, 4, 4, Color.WHITE);
canvas.drawText("GAME OVER", 200, 150, Color.RED, 4);   // 4 times bigger
int width = canvas.getTextWidth("GAME OVER", 4);        // for centering text
```

The letters come from the image `engine/src/pge/gfx/font.png`. It has English letters, digits and punctuation. Letters like `é` or `ç` show up as a box. Adding them is one of the exercises in section 11.

5. Sound and music

Playing sounds

Put WAV files in your game's `assets` folder. Java cannot read MP3 or OGG files.

```
// in init(), because loading takes time:
Sound jump = Sound.load("assets/jump.wav");

// in update(), when the player jumps:
jump.play();
```

If a sound is played again before it has finished, the second one plays at the same time as the first. Up to four copies can play at once, so picking up coins quickly does not cut the earlier coin sounds short.

Music is a sound that repeats:

```
Sound music = Sound.load("assets/music.wav");
music.setVolume(0.5);    // half as loud
music.loop();            // plays again and again, until music.stop()
```

To add a mute key, use `Sound.setMasterVolume(0)` to turn every sound off and `Sound.setMasterVolume(1)` to turn them back on. Have a look at `games/platformer-scenes/Sounds.java`. It loads every sound once, and then any part of the game can play one with, for example, `Sounds.jump.play()`.

You can find or make free sounds on the web. jsfxr makes retro sound effects in the browser: press a button such as "Jump" or "Pickup", then export a WAV file. Audacity can edit sounds and convert other formats to WAV.

Making sounds with code

A sound is stored as a list of numbers between -1 and 1. There are 22,050 numbers for every second of sound. A picture is also a list of numbers (one colour per pixel), so the idea is the same. The `Synth` class makes these lists for you:

```
// a "bwoop" that slides from 260 Hz up to 620 Hz in 0.16 seconds, at 40% volume
Sound jump = Sound.fromSamples(Synth.tone(Synth.SQUARE, 260, 620, 0.16, 0.4));

// two notes, one after the other
Sound coin = Sound.fromSamples(Synth.join(
        Synth.note(Synth.SQUARE, "B5", 0.06, 0.3),
        Synth.note(Synth.SQUARE, "E6", 0.2, 0.3)));
```

Hz (hertz) measures how high a sound is. More hertz means a higher sound.

There are five wave shapes, and each one sounds different:

Wave	Sound
`SQUARE`	buzzy, the classic video game sound
`TRIANGLE`	soft, good for bass
`SINE`	pure and gentle, like a whistle
`SAW`	harsh, good for alarms
`NOISE`	a hiss, good for explosions and drums

Notes have names like `"C4"` (middle C on a piano), `"F#5"` or `"Bb3"`. `Synth.mix(a, b)` plays two sounds at the same time.

`games/demo` makes all its sounds this way and has no sound files at all. `tools/MakeSounds.java` makes the platformer's sound effects and its music. To make music, it starts with silence and adds each note at the right moment, using `Synth.addInto`. Change a note, run it again and listen to the result. To keep a sound you made, save it with `sound.save("assets/mysound.wav")`.

If a computer has no working sound output, PGE prints a warning and the game runs without sound. The game does not crash.

6. Levels, walls and the camera

Tile maps

A level is often built from small square pictures called *tiles*, arranged in a grid. PGE stores a level as a text file of numbers, with one line for each row of the grid. This kind of file is called CSV (comma-separated values):

```
-1,-1,-1,-1
-1, 3, 3,-1
 0, 0, 0, 0
```

Each number says which picture of a sprite sheet goes in that place. This sprite sheet is called the *tile set*. `-1` means the place is empty.

```
SpriteSheet tiles = SpriteSheet.load("assets/tiles.png", 16, 16);
TileMap level = TileMap.load("assets/level1.csv", tiles);
level.setSolid(0, true);    // tile 0 is a wall
level.setSolid(3, true);

// in draw(), with the camera on:
level.draw(canvas);
```

Typing numbers by hand is slow. The free Tiled editor lets you paint levels with the mouse. Use File > Export As > CSV to save them. It writes one file for each layer, in exactly the format above.

Stopping at walls

You can ask the level whether a rectangle touches a wall:

```
level.overlapsSolid(x, y, width, height)
```

Here is a simple way to move a character without ever going through a wall: move one pixel at a time, and before each step, check whether the next position is free. The method `moveHorizontally` in `games/platformer/Main.java` does exactly this in about 15 lines. Read it carefully, because the platformer depends on it.

Gravity is a falling speed that gets a little bigger every update:

```
speedY = speedY + GRAVITY;
moveVertically(speedY);
```

The camera

```
Camera camera = new Camera(Engine.getWidth(), Engine.getHeight());
// never show anything outside the level
camera.setLimits(level.getWidth(), level.getHeight());
// move gently instead of jumping
camera.setSmoothness(0.15);
// ignore small movements, such as jumps
camera.setDeadZone(40, 160);

// in update():
camera.follow(heroX + 8, heroY + 8);    // follow the middle of the hero

// in draw():
camera.applyTo(canvas);
level.draw(canvas);
...
canvas.resetCamera();
```

The *dead zone* is a box in the middle of the screen. The hero can move around inside it without the camera moving. Without it, the whole screen would move up and down every time the hero jumps.

7. Bigger games: scenes and game objects

With one `Game` class, all the code goes in one file. That works for small games, but in a big game the file gets long and hard to follow. So PGE has a second way to build a game, which splits it into smaller pieces:

- A `Scene` is one screen of the game, such as the menu, a level or the game over screen.
- A `GameObject` is one thing in a scene, such as the hero, a slime or a coin.

Each object has its own `update()` method, where it takes care of itself. The scene updates and draws all its objects for you.

`games/platformer` and `games/platformer-scenes` are the same game written in the two different ways. Compare them. For example, the code that moves the hero is in the middle of `Main.java` in the first one, and in `Hero.java` in the second.

A game object

```
public class Coin extends GameObject {

    public Coin(double x, double y, SpriteSheet sprites) {
        super(x, y);
        setName("coin");
        setAnimation(new Animation(sprites.getFrames(8, 4), 7));
    }

    @Override
    public void update() {
        LevelScene level = (LevelScene) getScene();
        if (overlaps(level.getHero())) {
            remove();
            level.coinCollected();
        }
    }
}
```

You want to	Use
give the object a picture	`setSprite(bitmap)` or `setAnimation(animation)`. The scene draws it for you.
move it	`move(dx, dy)`, `setPosition(x, y)`, `getX()`, `getY()`
check if two objects touch	`overlaps(other)`. Use `setHitbox(...)` to make the touching area smaller than the picture.
find other objects	`getScene().find("hero")`, `getScene().findAll("slime")`
take it out of the game	`remove()`
draw it in front of others	`setDepth(10)`. Objects with a higher depth are drawn on top.

`getScene()` gives you a plain `Scene`. To call methods you wrote yourself, like `coinCollected()`, you need to *cast* it to your own scene class first: `(LevelScene) getScene()`.

A scene

```
public class LevelScene extends Scene {

    @Override
    public void start() {                       // runs once, to build the scene
        addLayer(TileMap.load("assets/level1.csv", tiles));
        add(new Hero(32, 300, sprites));
        add(new Coin(100, 280, sprites));
        addUI(new Label("Collect the coins!", 4, 4));
    }

    @Override
    public void update() {                      // after all the objects
        getCamera().follow(hero.getCenterX(), hero.getCenterY());
    }
}
```

Start the game with a scene. You can switch to another scene at any time:

```
Engine.run(new TitleScene(), 640, 360, "My Game");
Engine.setScene(new LevelScene());
```

Menus and buttons

Buttons, text and other screen elements are drawn on top of everything else. They use screen positions, so the camera does not move them:

```
Button play = new Button("Play", 270, 170, 100, 24);
addUI(play);

// in update():
if (play.wasClicked()) {
    Engine.setScene(new LevelScene());
}
```

There are also `Box` (a rectangle to put things on), `Label` (text, which can be made bigger, centred or given a shadow) and `Picture` (a small image, such as a heart for each life). You can hide or show any of them with `setVisible(...)`. `TitleScene.java` is a complete menu you can copy from.

8. Handing in your game

```
java Build.java package mygame
```

This creates a folder `dist/mygame/` and a zip file `dist/mygame.zip` with the same contents:

- `mygame.jar`, which holds the engine and your game in one file
- `assets/`, your pictures, sounds and levels
- `run.sh` and `run.bat`, which start the game on macOS/Linux and on Windows
- `README.txt`, which explains how to start the game

Whoever runs the game also needs Java 25. Check what your course wants you to hand in. Some courses want the source code instead, which is everything in `engine/src` and `games/mygame`.

9. When something goes wrong

When the game crashes, the window closes and the terminal shows a *stack trace*: a list of the methods that were running when the error happened. Read it from the top. The first line says what went wrong. The first line that mentions one of your own files tells you where it happened.

Message or problem	What usually causes it
`Could not find the image 'assets/hero.png'`	The name is spelled wrong, or the file is not in your game's `assets` folder. Capital letters matter: `Hero.png` is not the same as `hero.png`.

Message or problem	What usually causes it
`NullPointerException`	A variable was never given a value. For example, you forgot to load a picture in `init()`, or you called `find("hero")` but no object has that name.
`drawBitmap was given null`	The picture was never loaded.
Nothing moves	You changed the position in `draw()` instead of `update()`, or an `if` in `update()` stops the code from running.
Something keeps moving by itself	A value that should start again at zero every update (like `runDistance`) is a field instead of a local variable.
A jump goes down instead of up	`y` gets bigger going down, so up means a negative speed.
Things are in slightly the wrong place	Dividing two `int`s throws away the remainder: `7 / 2` is `3`. Write `7 / 2.0` to get `3.5`.
Two strings look the same but are not equal	Compare strings with `a.equals(b)`, not `a == b`.
The animation shows only its first picture	You forgot to call `animation.update()` in `update()`. (Game objects do this for you.)
`is not a sound format Java understands`	The file is MP3 or OGG. Convert it to WAV, for example with Audacity.
The game freezes for a moment when a sound plays	The sound is being loaded in `update()`. Load each sound once in `init()` or `start()`, and only call `play()` in `update()`.
`Engine.setScene only works in games started with a Scene`	Your game started with `Engine.run(new Main(), ...)`, and `Main` is a `Game`, not a `Scene`. Start the game with a scene instead.
`Compilation failed`	The lines above this message show the file, the line number and what Java expected. Fix the first error first, because it often causes the others.

10. Inside the engine

The engine is in `engine/src/pge`. It is about 4,000 lines long, and a lot of that is comments. If you want to read it, this order works well:

File	What you will learn from it
`Game.java`	the three methods every game has
`Engine.java`	the game loop: running updates at a fixed speed and measuring time
`gfx/Canvas.java`	how drawing works: every shape is just numbers written into an array. Includes the classic algorithms for drawing lines and circles.
`gfx/Color.java`	how a colour is stored in one `int`

File	What you will learn from it
`gfx/Bitmap.java`, `SpriteSheet.java`	how pictures are stored and loaded, and how a sprite sheet is cut into pieces
`gfx/Font.java`	how a font is just another sprite sheet
`input/Input.java`	how the engine knows a key was "pressed just now", and how two threads share data safely
`Window.java`	the one place where our pixels reach the screen
`world/TileMap.java`	how a grid is stored in a normal array, and how the map draws only the tiles you can see
`world/Camera.java`	the small formulas behind smooth camera movement
`scene/Scene.java`, `GameObject.java`, `SceneRunner.java`	how objects are organised, and how to change a list safely while you are looping over it
`ui/*.java`	inheritance and abstract classes: every screen element is a kind of `UIElement`
`audio/Synth.java`	how sound is made from numbers, and how note names become frequencies
`audio/Sound.java`	how Java plays sound, and what the engine does when there is no sound device

What happens in one frame

1. `Engine` checks whether it is time for the next update.
2. `Input.update()` records what the keyboard and mouse did since the last update.
3. Your `update()` runs. In a game with scenes, the buttons update first, then every object, then the scene.
4. Your `draw(canvas)` runs. Every drawing method writes colours into an array of `int`s called `pixels`.
5. `Window.present()` shows that array in the window, made bigger to fit.

So the picture on the screen is an array of 230,400 numbers (640 x 360). Every one of them was written there by Java code you can read.

11. Extending the engine

The best way to understand the engine is to change it. The exercises below go from small to big. Each one tells you where to start and gives you a hint.

When you add something to the engine, write it in the same style as the rest: simple classes, private fields with getters and setters, comments that explain *why* the code does what it does, and error messages a beginner can understand. Run `java Build.java` after every change. It compiles the engine and reports any warnings.

Warm-ups

- More colours. In `gfx/Color.java`, choose some names and add them as constants.
- Triangles. Add `drawTriangle` and `fillTriangle` to `gfx/Canvas.java`. An outline is three calls to `drawLine`. To fill it, go through the triangle one row at a time, work out where the edges cross that row, and fill the part in between with `fillSpan`.
- Accented letters. The font is drawn by `tools/MakeAssets.java`. Add a row of letters for `á é í ó ú ã õ ç`, then change `Font.java` so it can find them. At the moment it only knows the characters with codes 32 to 127.
- A pause key. In your game, when P is pressed, skip everything in `update()` except the check for P. Show "PAUSED" in `draw()`.

Medium

- Camera shake. In `world/Camera.java`, add a method `shake(strength, updates)`. While the camera is shaking, `getX()` and `getY()` add a small random number. It looks great for explosions.
- Bigger sprites. Add `drawBitmap(bitmap, x, y, scale)` to `Canvas.java`. Look at how `drawText` does it: every pixel of the picture becomes a square of `scale` by `scale` pixels.
- Particles. Make a class `Particle extends GameObject` with a speed, gravity and a timer. When the timer reaches zero, the particle calls `remove()`. Create 20 particles when a slime is stomped.
- A high score. Save it with `Files.writeString(Path.of("highscore.txt"), "" + score)`. Read it back with `Files.readString(...)` and `Integer.parseInt(...)`. Think about what should happen the first time, when the file does not exist yet.
- Moving platforms. Make a `GameObject` that moves back and forth. The hard part is that the hero should ride along on it. Hint: if the hero is standing on the platform, move the hero by the same amount as the platform.

Sound

- Fading music. Add `fadeTo(volume, updates)` to `Sound`. It changes the volume a little in every update until it reaches the new volume. Use it to fade the music out on the game over screen.
- Volume by distance. A slime should sound quieter when it is far away from the hero. Work out the distance between them, and turn it into a volume between 0 and 1.
- Different pitches. Hearing exactly the same jump sound every time gets boring. Make a few `Synth` versions that are slightly higher or lower, and pick one at random. For sounds loaded from files it is harder: you have to play the numbers a little faster or slower, for example by reading every 1.1th number instead of every one.
- Left and right speakers. At the moment every sound comes equally out of both speakers. Change `Sound.fromSamples` so it takes two arrays, one for the left speaker and one for the right. Then add a `pan` setting from -1 (left only) to 1 (right only).
- Music written as text. Write a tune as text, one step per line, like `C4 E5 –`. Then turn it into sound with `Synth`, the way `MakeSounds.music()` does with arrays.

- Long music. A `Sound` keeps the whole recording in memory. That is fine for a minute of music, but wasteful for ten. `javax.sound.sampled.SourceDataLine` can play a file while it is still reading it, a little at a time. It needs its own thread.

Bigger

- See-through colours. At the moment a pixel is either drawn or not drawn. Keep the PNG's transparency value (0 to 255, called *alpha*) in `Bitmap`. Then, in `Canvas.drawBitmap`, mix the two colours with `result = (source * alpha + destination * (255 - alpha)) / 255`, separately for red, green and blue. Use F3 to measure how much slower it gets.
- Animated tiles, like water with moving waves. Tell `TileMap` that some tile numbers are animations, and draw the current picture of the animation instead.
- Rotation. Add `drawBitmapRotated(bitmap, x, y, angle)`. Hint: go through every pixel of the area on the screen, turn its position backwards by the angle, and look up which pixel of the picture lands there. If you do it the other way round (turning each picture pixel forwards), you get holes. Try to work out why.
- Tiled's own file format. Tiled can also save `.tmx` files, which keep all the layers and objects of a level in one XML file. Java can read XML with `javax.xml.parsers`.